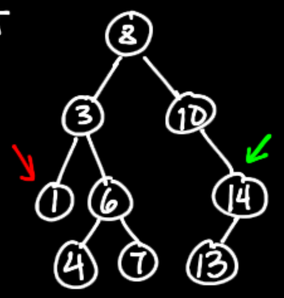
① Priority Queue

Items are given a priority value, and trees offer good performance

BST

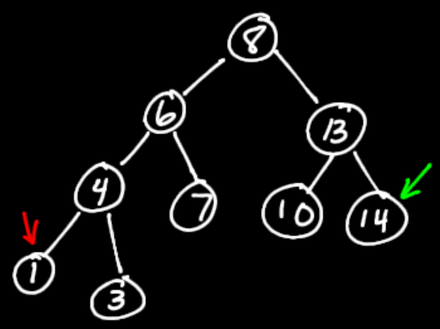


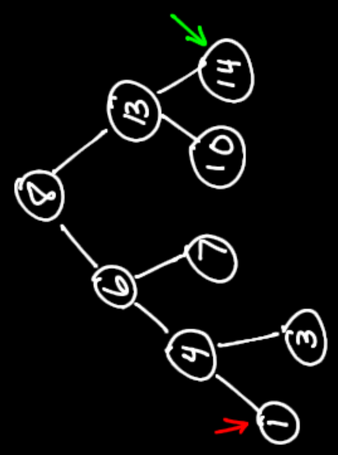
Decide what represents higher priority

- if larger values, then right most value
- if smaller values, then left most value

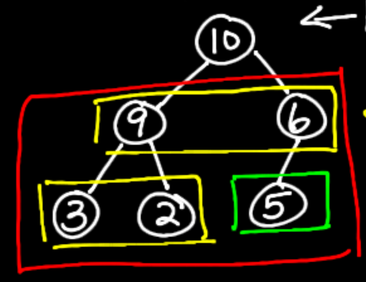
Advantages? Disadvantages? Performance?

Balanced BST





→ ② Max heap



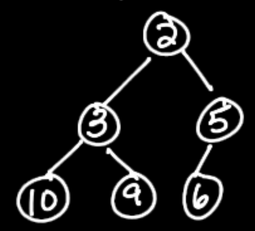
← Highest priority

All lower priority

Does not follow BST property

"Last node" - right most node

Min heap



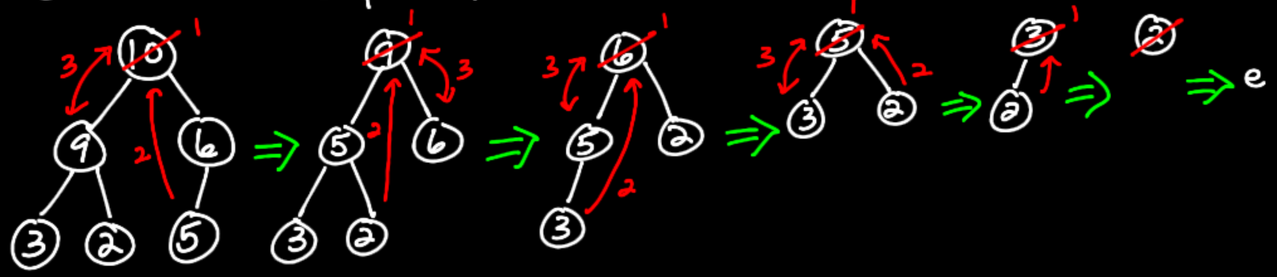
← Highest priority

Increasing values

Lower priority

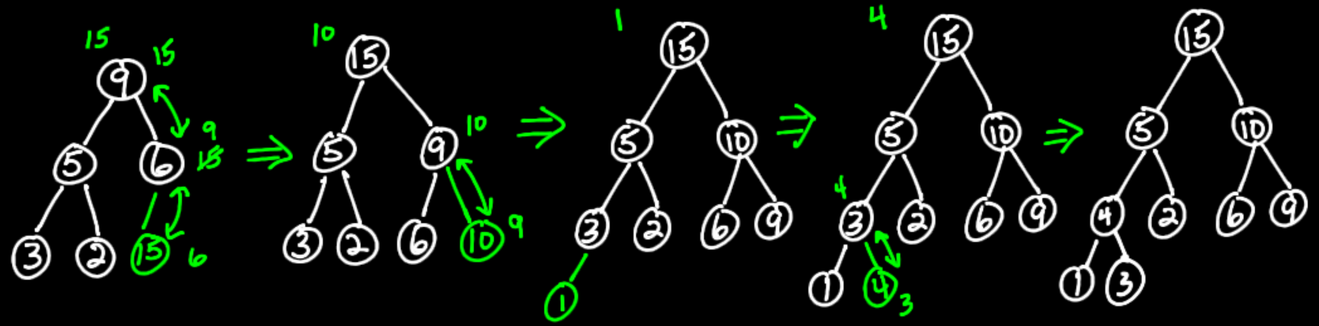
③ Heap Delete

① remove root ② place last node as root ③ "trickle down"



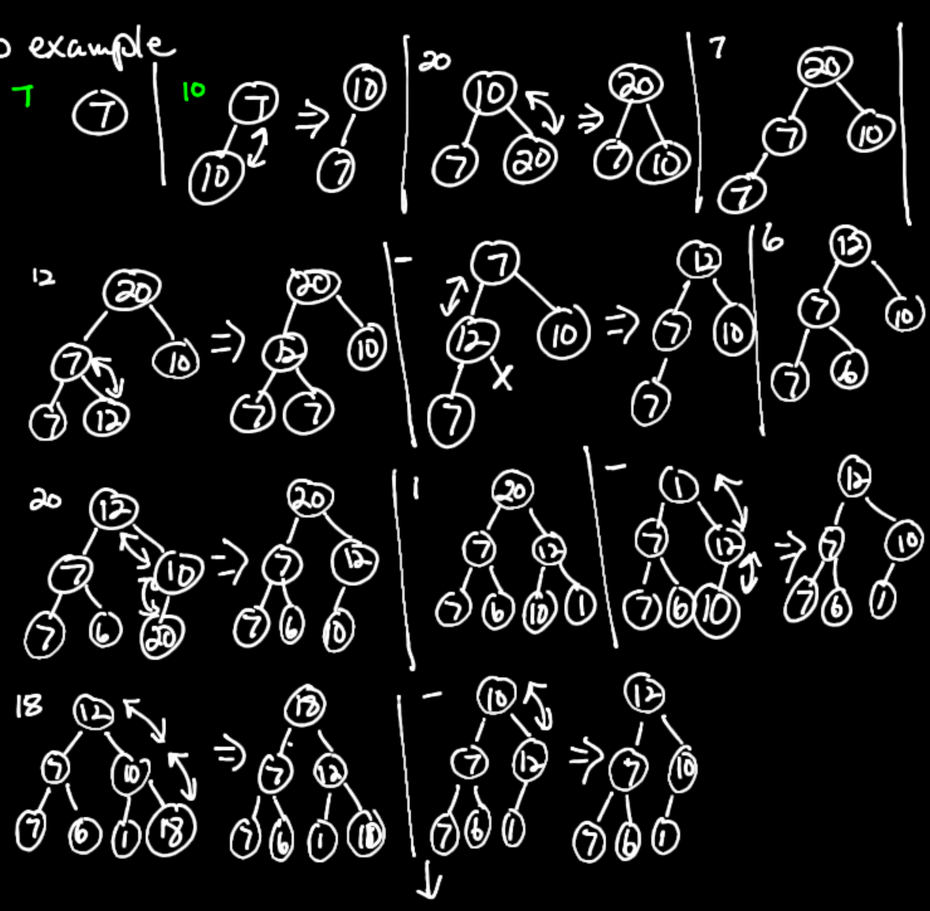
Heat Insert

① place value as the last node ② "Trickle up"



→ ④ Max heap example

Insert 7
Insert 10
Insert 20
Insert 7
Insert 12
Delete
Insert 6
Insert 20
Insert 1
Delete
Insert 18
Delete

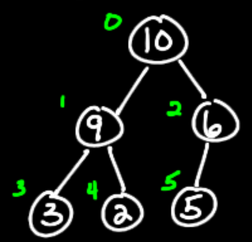


⑤ Heap as an array

Parent (i-1)/2

left child 2i+1

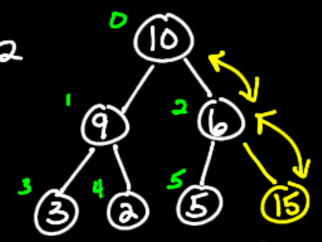
right child 2i+2



0 1 2 3 4 5

10 9 6 3 2 5

Insert 15



0 1 2 3 4 5

10 9 6 3 2 5 15

parent = (6-1)/2 = 2

10 9 15 3 2 5 6

parent = (2-1)/2 = 0

15 9 10 3 2 5 6

15 9 10 3 2 5 6

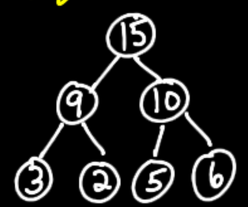
left = 2(0)+1 = 1

right = 2(0)+2 = 2



left = 2(1)+1 = 3

right = 2(1)+2 = 4



i=3 i=4 i=5 i=6

left = 2(2)+1 = 5

right = 2(2)+2 = 6

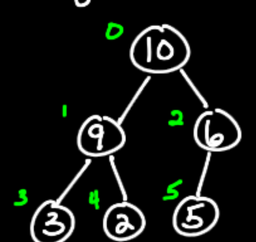
→

⑥ Heap as an array

Parent (i-1)/2

left child 2i+1

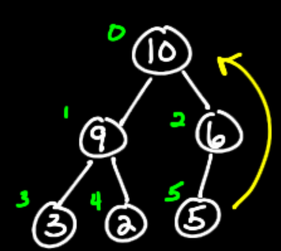
right child 2i+2



0 1 2 3 4 5

10 9 6 3 2 5

Delete



0 1 2 3 4 5

10 9 6 3 2 5

Last node = size - 1 = 5

5 9 6 3 2

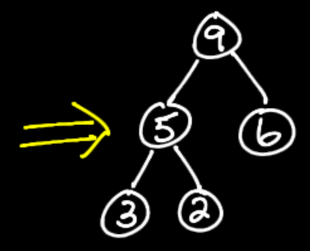
Check for larger 2i+1 or 2i-1 w/ i=0

[2i+1 = 2(0)+1] > [2i+2 = 2(0)+2]

9 5 6 3 2

now w/ i=1

[2(1)+1] and [2(2)+2] are both < 9

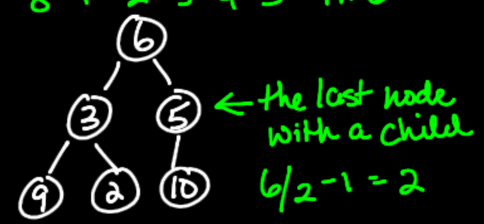


9 5 6 3 2

⑦ Heap sort

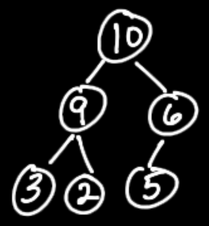
6 3 5 9 2 10

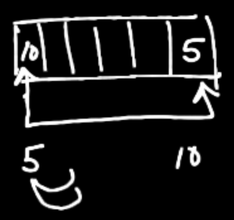
0 1 2 3 4 5 n=6



← the last node with a child

6/2 - 1 = 2





Starting w i=2

rebuild index 2

2(2)+1 = 5 (10)

2(2)+2 = 6 (none)

10 > 5, swap

6 3 10 9 2 5

rebuild index 1

2(1)+1 = 3

2(1)+2 = 4

9 > 2 > 3, swap

6 9 10 3 2 5

rebuild index 0

2(0)+1 = 1

2(0)+2 = 2

10 > 9 > 6, swap

10 9 6 3 2 5

Now that we have the heap, we are ready to sort

⑧ 10 9 6 3 2 5

take the first value and swap, then trickle down

5 9 6 3 2 | 10

9 5 6 3 2 | 10

---

9 5 6 3 2 | 10

2 5 6 3 | 9 10

6 5 2 3 | 9 10

---

6 5 2 3 | 9 10

3 5 2 | 6 9 10

5 3 2 | 6 9 10

---

5 3 2 | 6 9 10

2 3 | 5 6 9 10

3 2 | 5 6 9 10

---

3 2 | 5 6 9 10

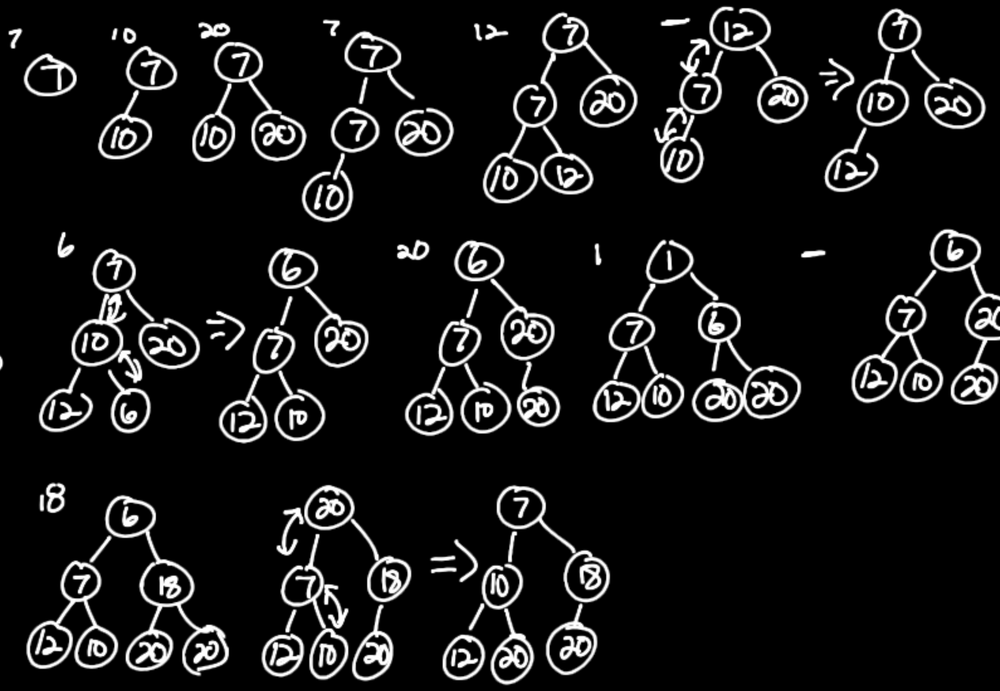
2 | 3 5 6 9 10

---

2 3 5 6 9 10

min heap

Insert 7
Insert 10
Insert 20
Insert 7
Insert 12
Delete
Insert 6
Insert 20
Insert 1
Delete
Insert 18
Delete



0 1 2 3 4 5 6 7
87 21 60 50 29 38 44 25

n=8
2(3)+1=7
2(2)+1=5
2(2)+2=6
2(1)+1=3
=4

87 50 60 21 29 38 44 25

25 50 60 21 29 38 44 | 87

2(0)+1=5
+2=6

60 50 25 21 29 38 44 | 87

2(2)+1 5
2(2)+2 6

60 50 44 21 29 38 25 | 87

25 29 38 21 | 44 50 60 87

25 50 44 21 29 38 | 60 87

38 29 25 21 | 44 50 60 87

50 25 44 21 29 38 | 60 87

21 29 25 | 38 44 50 60 87

50 29 44 21 25 | 60 87

29 21 25 | 38 44 50 60 87

38 29 44 21 25 | 50 60 87

25 21 | 29

44 29 38 21 25 | 50 60 87

21 | 25 29 38 44 50 60 87

25 29 38 21 | 44 50 60 87